

MODELO DE DATOS

Sistema de Gestión de Ventas

Problema 4 · Django REST Framework + PostgreSQL
8 Entidades · 7 Relaciones · Schema: ventas_schema

Tecnológico en Desarrollo de Software

1. Descripción del Problema

Una empresa comercial requiere un sistema para administrar sus operaciones de ventas de extremo a extremo. El sistema cubre la gestión de clientes, proveedores, sucursales, productos, pedidos, el detalle de cada pedido, facturas y pagos.

El ciclo de negocio comienza cuando un cliente realiza un pedido en una sucursal específica; ese pedido incluye uno o más productos con su cantidad y precio histórico. Al confirmar el pedido se genera automáticamente una factura (relación 1:1), y sobre esa factura se registran los pagos parciales o totales que el cliente realiza por distintos medios (efectivo, transferencia, tarjeta de crédito/débito, cheque, PSE).

La solución se implementa como una API REST construida con Django y Django REST Framework, conectada a una base de datos PostgreSQL organizada en el schema dedicado ventas_schema. El modelo aplica normalización en 3FN, soft delete, auditoría de usuarios y cálculos automáticos de totales, garantizando integridad referencial y trazabilidad completa de las transacciones comerciales.

2. Diagrama Entidad-Relación

El siguiente diagrama resume las 8 entidades del modelo, sus atributos principales y las claves foráneas que establecen las relaciones entre tablas.

Entidad	Atributos principales	Relaciones (FK)
PROVEEDORES	PK id, nit (UNIQUE), nombre, contacto, email, telefono, ciudad, activo	—
PRODUCTOS	PK id, codigo, nombre, precio, stock, stock_minimo, unidad_medida, activo	FK proveedor_id → PROVEEDORES
SUCURSALES	PK id, codigo (UNIQUE), nombre, direccion, ciudad, es_principal, activo	—
CLIENTES	PK id, tipo/num_documento (UNIQUE), nombre, apellido, email (UNIQUE), telefono, ciudad, activo	—
PEDIDOS	PK id, numero_pedido (UNIQUE), fecha_pedido, estado, total, activo	FK cliente_id → CLIENTES FK sucursal_id → SUCURSALES
DETALLE_PEDIDOS	PK id, cantidad, precio_unitario, descuento, subtotal UNIQUE(pedido_id, producto_id)	FK pedido_id → PEDIDOS (CASCADE) FK producto_id → PRODUCTOS
FACTURAS	PK id, numero_factura (UNIQUE), subtotal, iva, total, estado, activo	FK pedido_id → PEDIDOS (UNIQUE → 1:1)
PAGOS	PK id, numero_pago (UNIQUE), monto, metodo_pago, estado, referencia, activo	FK factura_id → FACTURAS

Nota: PK = Clave Primaria · FK = Clave Foránea · UNIQUE = Restricción de unicidad · CASCADE = Eliminación en cascada

3. Relaciones Implementadas

El modelo cuenta con 7 relaciones entre entidades, implementadas como claves foráneas en PostgreSQL. A continuación se detalla cada relación, su cardinalidad y el mecanismo de implementación.

Relación	Tipo	Descripción
Producto → Proveedor	N : 1	Cada producto pertenece a un único proveedor. Un proveedor puede suministrar múltiples productos. Se implementa con el campo proveedor_id (NOT NULL) en la tabla productos.
Pedido → Cliente	N : 1	Un cliente puede realizar muchos pedidos. Cada pedido pertenece a un único cliente, registrado mediante cliente_id (NOT NULL) en la tabla pedidos.
Pedido → Sucursal	N : 1	Cada pedido es atendido por una sucursal específica. Una sucursal puede gestionar múltiples pedidos, mediante sucursal_id (NOT NULL) en la tabla pedidos.
DetallePedido → Pedido	N : 1	Un pedido tiene múltiples líneas de detalle. Cada detalle pertenece a un único pedido mediante pedido_id con CASCADE al eliminar, garantizando integridad referencial.
DetallePedido → Producto	N : 1	Cada línea de detalle referencia un producto. El precio se almacena históricamente en precio_unitario del detalle, preservando el valor al momento de la venta.
Factura → Pedido	1 : 1	Cada pedido genera exactamente una factura y cada factura corresponde a un único pedido. Se implementa con restricción UNIQUE sobre pedido_id en la tabla facturas.
Pago → Factura	N : 1	Una factura puede recibir múltiples pagos parciales o uno total. Cada pago referencia una única factura mediante factura_id, permitiendo el seguimiento por cuotas.

4. Justificación del Modelo

Las decisiones de diseño del modelo de datos responden a principios de normalización, integridad referencial, trazabilidad y rendimiento, detallados a continuación.

1. Normalización en Tercera Forma Normal (3FN)

El modelo se encuentra en 3FN. Los datos del proveedor no se repiten en la tabla de productos; los datos del cliente no se repiten en pedidos. El precio unitario se almacena en detalle_pedidos (no en productos) para preservar el historial de ventas con independencia de cambios futuros en el catálogo.

2. Soft Delete (borrado lógico)

Todas las tablas implementan el campo activo (BOOLEAN, DEFAULT TRUE) en lugar de eliminar registros físicamente. Esto preserva la integridad histórica de facturas, pedidos y pagos, y permite auditorías y reversiones sin pérdida de trazabilidad.

3. Auditoría de Usuarios

Los campos creado_por_id y modificado_por_id (FK → auth_user) permiten rastrear qué usuario del sistema realizó o modificó cada registro, cumpliendo requisitos de control en entornos multiusuario con roles diferenciados (Administrador, Vendedor, Bodega).

4. Cálculos Automáticos y Consistencia

El subtotal en detalle_pedidos se calcula como: $\text{cantidad} \times \text{precio_unitario} \times (1 - \text{descuento}/100)$. El total en pedidos se obtiene sumando sus detalles, y el total en facturas aplica el IVA sobre el subtotal. Todos estos recálculos se ejecutan en el método save() de los modelos Django, garantizando consistencia sin depender de lógica en el cliente.

5. Índices y Rendimiento

Se crean índices en todas las claves foráneas y en los campos utilizados frecuentemente como filtros: estado, activo, email, numero_documento y fecha_pedido. Esto optimiza las consultas más frecuentes como listado de pedidos por cliente, filtrado por estado de factura y búsqueda de clientes activos.

6. Restricción UNIQUE Compuesta en Detalle

La restricción UNIQUE(pedido_id, producto_id) en detalle_pedidos garantiza que un mismo producto no se registre dos veces en el mismo pedido, previniendo duplicados a nivel de base de datos independientemente de la lógica de aplicación.